DEEC
DEPARTAMENTO DE ENGENHARIA ELETROTÉCNICA E DE COMPUTADORES
TÉCNICO LISBOA

Licenciatura em Engenharia Electrotécnica e de Computadores (LEEC)

Algoritmos e Estruturas de Dados - 2026/2027
Aula de Laboratório #05 - 9 a 13 de Novembro de 2026
Grafos: Representações, Conversões e Travessia

Objectivos

Neste laboratório abordam-se os tópicos da representação abstracta (ADT), conversão entre representações e travessia de grafos. É fornecido um conjunto de ficheiros base (`graph.h`, `graph.c`, `main.c`, `LinkedList.h`, `LinkedList.c`, `defs.h` e `Makefile`) que definem um Tipo Abstracto de Dados (*Abstract Data Type* – ADT) para representar grafos não direccionados e ponderados (`Graph`).

Ao longo do laboratório, pretende-se completar as funções em falta em `graph.c` e o programa cliente em `main.c` (invocado através do executável `lab05 <nome_grafo>.adj`), de modo a:

1. Ler um grafo a partir de um ficheiro em formato de **matriz de adjacências** (`.adj`), construir a sua representação em matriz e calcular propriedades básicas (densidade e grau máximo);
2. Converter a representação em matriz de adjacências para **lista de adjacências** e escrever o ficheiro `<nome_grafo>.ladj`;
3. Converter a representação em lista de adjacências para um **vector de arestas** e escrever o ficheiro `<nome_grafo>.edge`;
4. Realizar uma **travessia em profundidade** (*Depth-First Search* – DFS) sobre o grafo para determinar a ordem de visita dos vértices e o número de componentes conexas.

1 Representações de um Grafo e Interface ADT

A tabela abaixo ilustra as três representações de um mesmo grafo ponderado e não direccionado com $V = 6$ vértices (numerados de 0 a $V - 1$) e $E = 5$ arestas, cuja densidade é $2E/V = 1.667$ e o grau máximo é 3:

Matriz de adjacências	Lista de adjacências	Vector de arestas
6 0 2 1 3 0 0 2 0 0 0 0 0 1 0 0 4 5 0 3 0 4 0 0 0 0 0 5 0 0 0 0 0 0 0 0 0	6 1:2 2:1 3:3 -1 0:2 -1 0:1 3:4 4:5 -1 0:3 2:4 -1 2:5 -1 -1	6 5 0 1 2 0 2 1 0 3 3 2 3 4 2 4 5
grafo6.adj	grafo6.ladj	grafo6.edge

Estude os ficheiros `graph.h` e `graph.c` fornecidos:

- Em `graph.h` encontram-se definidos os tipos abstractos opacos `Edge` (`typedef struct edge Edge;`) e `Graph` (`typedef struct graph Graph;`), bem como os protótipos das operações disponibilizadas sobre esses mesmos tipos.
- Em `graph.c`, a estrutura `struct edge` encapsula os extremos (`v`, `w`) e o peso (`wt`), e a estrutura `struct graph` encapsula o número de vértices (`V`), o número de arestas (`E`), a matriz de adjacências (`int **adj`) e o vector de listas de adjacências (`LinkedList **ladj`), em que cada elemento da lista aponta para uma estrutura `AdjNode` contendo o vértice adjacente `v` e o peso `wt`.

2 Leitura da Matriz de Adjacências e Propriedades (Passos 1 e 2)

Um ficheiro de entrada `<nome_grafo>.adj` contém na primeira linha o inteiro V (número de vértices), seguido de V linhas com V inteiros não negativos cada. Um valor positivo na posição (i, j) indica o peso da aresta entre i e

j, enquanto 0 indica ausência de aresta.

a) Em `graph.c`, complete a função `Graph *GRAPHreadMat(FILE *fp)` que lê o número de vértices V, inicializa o grafo através de `GRAPHinit(V)` e lê as $V \times V$ entradas da matriz, criando e inserindo as arestas de peso positivo com `EDGE(i, j, wt)`, `GRAPHinsertE(G, e)` e `EDGEdestroy(e)`.

b) Complete também em `graph.c` as funções que calculam propriedades do grafo a partir da matriz de adjacências (use a função `int GRAPHdegree(Graph *G, int v)`, que devolve o grau do vértice `v` como exemplo):

- `int GRAPHmaxDegree(Graph *G)`: devolve o grau máximo do grafo;
- `double GRAPHdensity(Graph *G)`: devolve a densidade do grafo ($2E/V$).

c) Compile e execute o código com os exemplos fornecidos e analise os resultados obtidos,

3 Conversão entre Representações (Passos 3 e 4)

a) **Matriz de Adjacências → Lista de Adjacências:** Complete em `graph.c` a função `void GRAPHmatToList(Graph *G)`, que percorre a matriz `G->adj` e, para cada posição (i, j) com peso positivo, cria um nó `newAdjNode(j, G->adj[i][j])` e insere-o de forma ordenada na lista `G->ladj[i]` usando `insertSortedLinkedList()` e a função de comparação `compareAdjNodes`.
Descomente o código correspondente ao **Passo 3** em `main.c` para converter o grafo e escrever o ficheiro de saída `<nome_grafo>.ladj` recorrendo à função já fornecida `GRAPHwriteList()`.
Execute o código com os exemplos fornecidos e analise os resultados.

b) **Lista de Adjacências → Vector de Arestas:** Complete em `graph.c` a função `int GRAPHlistToEdges(Edge *a[], Graph *G)`, que percorre as listas de adjacências `G->ladj[v]` para $0 \leq v < V$ e extrai todas as arestas do grafo para o vector de ponteiros `a[]`, devolvendo o número total de arestas guardadas. *Nota:* Como o grafo não é direccionado, cada ramo (v, w) surge tanto na lista de v como na de w; para evitar duplicados no vector, adicione a aresta `EDGE(v, w, wt)` apenas quando $v < w$.
Descomente o código correspondente ao **Passo 4** em `main.c` para alocar o vector `Edge **edges`, invocar `GRAPHlistToEdges()`, escrever o ficheiro `<nome_grafo>.edge` usando `GRAPHwriteEdges()` e libertar as arestas com `EDGEdestroy()`.
Execute o código com os exemplos fornecidos e analise os resultados.

4 Travessia de um Grafo – Procura em Profundidade (Passo 5)

Para visitar sistematicamente todos os vértices e arestas de um grafo, um dos algoritmos fundamentais estudados nas aulas teóricas é a **Procura em Profundidade** (*Depth-First Search* – DFS).

O algoritmo utiliza um vector `pre[]` (de dimensão V), inicializado a `-1` para indicar que nenhum vértice foi ainda visitado, e um contador `cnt` inicializado a `0` que regista a ordem em que cada vértice é descoberto:

```
void dfsR(Graph *G, int v, int pre[], int *cnt) {
    /* 1. Marcar o vertice v como visitado: pre[v] = (*cnt)++;
     * 2. Percorrer a lista de adjacencias G->ladj[v]:
     *    Para cada vertice adjacente w:
     *      Se w ainda nao foi visitado (pre[w] == -1),
     *      chamar recursivamente dfsR(G, w, pre, cnt);
     */
}
```

A função principal de travessia, `int GRAPHdfs(Graph *G, int pre[])`, inicializa `pre[v] = -1` para todos os vértices $0 \leq v < V$ e, de seguida, percorre todos os vértices v: sempre que encontra um vértice ainda não visitado (`pre[v] == -1`), incrementa o contador de componentes conexas e invoca `dfsR(G, v, pre, &cnt)`.

a) Implemente em `graph.c` as funções `dfsR()` e `GRAPHdfs()`.

b) Complete o **Passo 5** em `main.c` para invocar `GRAPHdfs(G, pre)` e imprimir no terminal o número de componentes conexas do grafo e a ordem de visita `pre[v]` de cada vértice. Verifique com o comando `make tv` (usando `valgrind`) que não existem fugas de memória.